开放原子开源基金会
OPENATOM FOUNDATION

OpenHarmony

2026开源鸿蒙大学生创新大赛

赛道一：系统与技术创新赛道

方舟智诊



面向 OpenHarmony 北向应用的智能化基础体验分析与优化 Agent

参赛团队 智能应用与教学赋能团队

参赛学校 广州商学院

仓库 github.com/fan-weibo/ArkPerf

目录
CONTENTS

PART 01
真实痛点与生态缺口

PART 02
方案与系统架构

PART 03
关键技术创新

PART 04
实测验证

PART 05
稳定性与工程规范

PART 06
团队与规划

痛点洞察　缺的不是数据，是结论

需求侧 3 条原声 × 供给侧 4 层缺口 · 逐条可溯源

三平台开发者原声

原声一 —— Gitee OpenHarmony Issue　**看不懂数据**

“性能 Profiler 工具采集出来 trace 数据，火焰图信息很多，对于新手开发者，很难快速定位业务。ArkTS 代码的卡顿点，需要花费大量时间学习工具的使用方法。”

原声二 —— CSDN 鸿蒙开发嫂文评论　**工具太散**

“静态检查、Profiler Trace、源码三份东西要分开看，一怎切 CodeLinter，一怎切 Profiler，再打开源码来回对照，来回切换特别耗时。”

原声三 —— 昇金社区　**AI 给的不能用**

“拿通用大模型生成鸿蒙 ArkTS 优化代码，经常输出普通 TS 代码，直接复制进去一堆语法报错，没法直接编译运行。”

需求结论：社区已反馈通用大模型给的鸿蒙优化代码直接编译报错，纯大模型路线走不通——测量与判定必须落在确定性代码里。

原声 × 缺口



逐条可溯源

生态缺口（四层能力）

① 采集数据

hdc 官方 CLI 已支持 ✓

② 自动判定

需自建：规则 + 统计检验

③ 自动报告

需自建：MD + JSON 双落盘

④ 自动化前后对比

需自建：tag 归档 + 回归检验

ArkPerf 补的正是 ②③④：判定、报告、前后对比

——把「能采数据」补成「能出结论」。

数据不缺，缺的是有人把数据翻译成结论和改法——三条原话指向同一个断点。

对标差异　比官方 Profiler 多一层闭环

vs DevEco Studio Profiler · 六维度

维度	DevEco Studio Profiler（官方 GUI）	ArkPerf	差异要点
使用形态	图形界面，人工点选与判读	TUI / 桌面对话驱动，Agent 自主执行	由人操作 → 由 Agent 执行
统计口径	展示曲线与数值	中位数 + 离散度 + p 值 + MDD	从看数值 → 给显著性结论
环境漂移	不自动对冲	锚点归一化，每轮同步测锚点	假阳性从根上消掉
问题判定	人看曲线下判断	规则引擎 + 三级分级，不自造阈值	判定可复用、可复算
优化闭环	需自行组织前后对比	baseline/optimized tag → compare 报告	闭环内建，三明治法校验
可集成性	GUI 工具，难脚本化	MCP 标准协议，13 工具可被任意 Agent 挂载	可进 CI、可被复用

① 使用形态展开见 P9/ P7（工作链路 • TUI 交互形态）；② 统计口径见 P15（Mann-Whitney U + MDD）；③ 环境漂移见 P14（锚点归一化）；④ 问题判定见 P17（规则引擎 + 三级分级）；⑤ 优化闭环见 P21/ P23；⑥ 可集成性见 P12（MCP 三层结构）。

官方工具解决「看得见」，我们补的是「判得准、可复算、能闭环」

系统架构　两层自研，底部只调 hdc

前端层
两个交付形态 · 两种壳复用

- **TUI 终端界面** — bubbletea v2
- **桌面端 Wails3 + Vue3** — 三栏：功能 / 聊天 / 工作区

↓

装配层
internal/app · 自研

配置 + 工具注册表 + MCP 连接 + 会话 → 共用 Session

↓

★ 自研内核层
internal/kernel · Go 1.26.5

Agent 内核：模型循环 · 工具注册表 · 会话 · 流式输出 · 指数退避重试 · 审批持久化 · 上下文预算

↓

本地工具层 • 协议层
internal/mcp · 自研

- **鸿蒙域 23**
- **通用 7**

MCP 服务器 → 本地工具投影

↓

★ 测量工具层
mcp-servers/ · Python 3.13 · 核心资产

- `cold_start`
- `cpu`
- `agent`
- `memory`
- `jank`

5 个 MCP 服务器 · 13 个工具

↓

设备层 • hdc
官方 CLI · 无侵入

`aa start -W · hidumper --mem-jsheap --gc · hilog(JankFrameMonitor / RSJankStats)`
`uitest · bm · OpenHarmony 模拟器 API 24（常驻不重启）`

技术栈

- Go 1.26.5
- Python 3.13 + mcp
- Wails3 · bubbletea v2 · Vue3
- hdc 3.2.0d
- 模拟器 API 24（常驻不重启）

补充说明

`internal/kernel/` 是完整实现，非半成品：14 个非测试 `.go` 文件覆盖全部内核能力（模型循环 / 工具注册表 / 会话 / 流式输出 / 指数退避重试 / 审批持久化 / 上下文预算），且均有对应 `_test.go`。

两层自研：Go 内核 + Python 测量层。底部 hdc 为官方 CLI——全程无侵入、不 root、不改系统。

自研边界　出结论的地方全部自己写

自研 6 项 / 复用 4 项

自研 6 项

- ✓ Agent Go 内核实现
- ✓ 测量工具层（5 服务器 / 13 工具、三层结构）
- ✓ 抗噪声判定方法学（锚点 / MW-U+MDD / 三态 / 噪声函数）
- ✓ 规则引擎 + 报告生成
- ✓ PerfLab 示例应用
- ✓ AGENTS.md 口径纪律

核心资产

复用 4 项

- ＋ 大模型 API（DeepSeek，可插拔）
- ＋ 开源 UI 库（bubbletea / Wails / Vue）
- ＋ MCP 协议标准
- ＋ 设备工具链（hdc / hvigorw，官方）

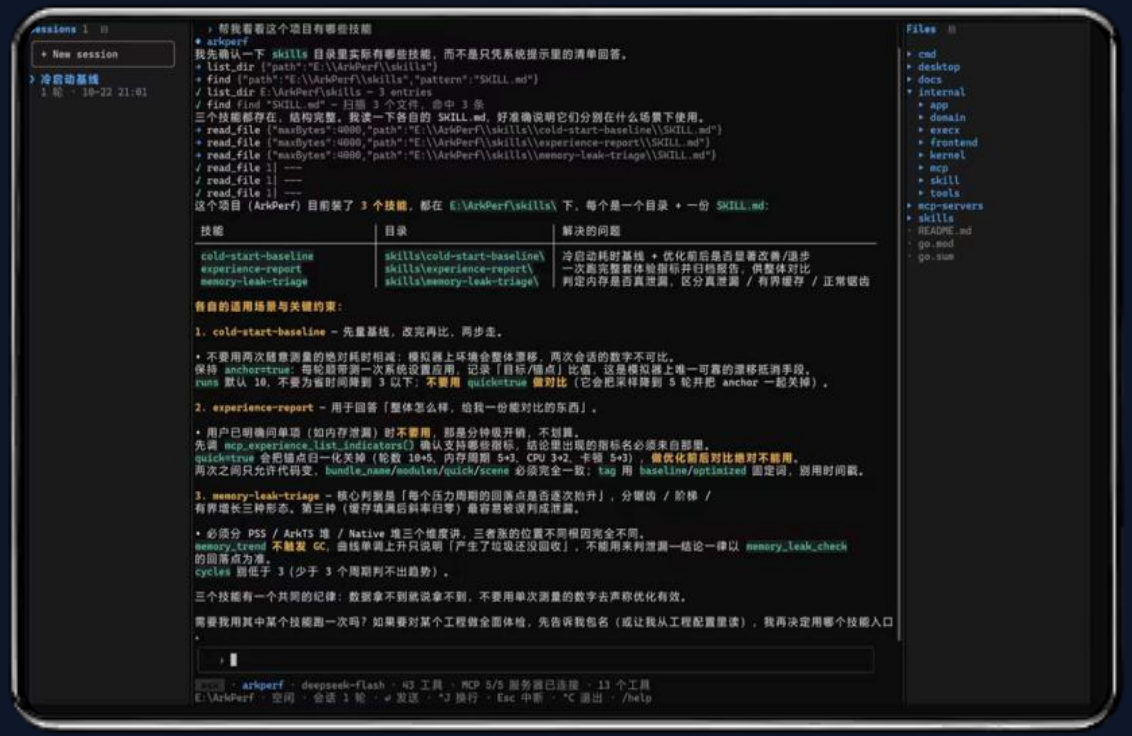
边界：

复用开源库是正常的工程决策，核心资产在测量与判定层

模型可插拔、UI 库与官方工具链复用，而测量与判定层——出结论的地方——全部自研。

交互形态　用户只说一句话，工具后台测

TUI 对话驱动 · 第一交付形态



一句话任务

对 com.example.perflab 做一次体验分析（快速模式），报告落盘，tag 用 baseline

Agent 自主完成

确认设备在线 → 全量采集 → 规则判定 → 报告落盘 (MD + JSON)

两种用法

交互式多轮追问（复用会话上下文深挖，如「内存测几个周期」）· **一次性任务**（演示 / CLI 脚本化）

会话回放包含工具调用在内 —— 一回放就会话丢上下文（「切个会话回放未来工具调用全没了」，实测被指出后补上）。

用户面对的是对话，不是命令；工具在背后「测」—— 门槛从学工具降到会描述。

内核层 自研 Go Agent 内核的结构与工作原理

主循环：一个 for 循环，四种停法

模型 → 工具调用 → 审批（分级 / 护栏）→ 执行 → 结果回灌 → 下一轮；final 停妥 · idle 连续 12 轮成功工具调用耗尽卡死 · turn-veto 撞 MaxTurns（200）· interrupted 被中断

「模型停妥了」和「撞上阀门了」是两件事 —— 区分它们是可观测性的基础。

上下文：先硬限化，再丢器官

工具输出（构建日志、测量报、星历图）是块大块，也是最先过的 —— 但删块删脚脖子于失忆，上一轮的数字还在里面。① 先把要紧的工程蛆点压点可用摘要，盯的回档的完整留存 ② 灯链剪笋，才吞繁回到整轮

硬规则：切最必须像在 user 消息上（否则留下七结果下轮的 tool_calls，端点最景 400）；当前轮一律不动。

安全：分级决定问不问，护栏决定不许做

路径三级：工作区内免审批 · 状态相关正审批 · 工作区外审批

状态领先于工作区 —— 否则「按 ArkPerf 自己的覆盖」会变成免死金牌，那正是最容易把自己弄坏的地方。

破坏性命令无问什么可兼判定，但趋总必须说明中哪条规则问 —— 只说「被拒绝」会让用户没办法过失。

可测与可移植：换台机器能跑，不接真模型也能测

ChatFunc 注入接缝：生产用真模型，测试用假实现，整个循环离线跑完，不烧钱，不依赖网络。

token 预算：CJK 1 字符 ≈ 1 token，ASCII 4 字符 ≈ 1 token；刻意不引 tokenizer，内核零依赖。

测量服务器与解释器按可执行位置完全解析 —— 只写绝对路径记，换个拉不起来、一台服务器都测不上。

主张

内核按「Agent 长期自主运行」的需要设计，不是把大模型 API 包一层

代码结构

14 个文件 · 按组分级 6 组 · 不按技术分层，按「删掉谁是同一件事」分

- **循环** loop.go · run.go
- **模型** provider.go · stream.go
- **上下文** conversation.go · prompt.go · template.go
- **安全** guard.go · approval.go
- **可靠** retry.go
- **基础** registry.go · config.go · session.go · types.go

分级与护栏放在同一文件 —— 它们是同一枚底牌的两面，分散到工具里迟早漏一名。

14 个文件、四条实现逻辑：循环能停 • 历史能压 • 危险能拦 • 行为能测

工作链路　一句话进，报告自动落盘

五步全自主 · 无需人工干预



① 自然语言输入	② 前置检查	③ 逐模块采集	④ 规则引擎归并	⑤ 落盘
描述目标 “对 com.example.perflab 做一次体验分析，把报告落盘，tag 用 baseline”	**四项自检校验** 设备在线 / 应用安装 / 工具链就绪 / 自动拉起	**四类指标定轮次** 冷启动 10 轮 + 焦点 → 内存 5 周期 + 强制 GC → CPU 3 次 → 卡顿 5 轮	**现象 → 原因 → 建议** 规则命中 + 统计判定 + 三分级	**MD + JSON 双格式** 原始测量序列随 JSON 一样保存，可离线重判 按规则添加注：掘到只有名字与描述进提示词，正文和图表描述相归给自己像 —— 「工具自描述一双」，技能给一份「手册」，会在丢不掉上下文。

链路本质：用户只描述目标（包名 + 场景 + tag），检查、采集、判定、报告全部由 Agent 自主完成——报告直接落盘为 Markdown + JSON 双格式。

场景直达设备：aa start --ps scene leakblockldiscover，配合 SceneHolder 静态类中转（绕开 .js ↔ .ets 的 import 边界），全程无人工 UI 导航。

口径：五步不是脚本硬编码，而是 Agent 依据工具描述自行编排——证据见下一页真实轨迹。

五步闭环里没有一步需要人手动点——这是「Agent」二字的实际含义。

实证轨迹　8 轮 9 调用，Agent 自己跑完

不是脚本预设，是自主编排



正确编排

确认设备在线 → 全量采集落盘 → 输出报告，三步按序完成。



自发质疑「0 问题」结论

独立复核 4 个工具：cpu_usage / memory_snapshot / cpu_trend / harmony_logs。



用 hilog 中 XPERF_SERVICE 时间戳

证明滑动负载确实注入到应用前台，排除后台冻结假阴性。



口径纪律生效

区分「未观测到」与「证明正常」，标注 CPU 样本不足、卡顿日志降级两处证据弱点。

01 **正确编排 · 8 轮对话**
ORCHESTRATION

02 **9 次工具调用**
TOOL CALLS

03 **总耗时 2 分 31 秒**
TOTAL TIME

04 **独立复核 4 个工具**
SELF CHECK

05 **留痕回放**
AUDIT TRAIL

证据链 · 留痕可回溯

证据：追加式会话审计 + run 摘要，全部工具调用留痕、逐轮可回放。

数据来源：ArkPerf 端到端会话审计记录

2:31 总耗时

8 轮 9 调用不是脚本跑出来的，是 Agent 自己编排、自己质疑、自己取证的结果。

8 轮 9 调用不是脚本跑出来的，是 Agent 自己编排、自己质疑、自己取证的结果。

工具覆盖　23 个鸿蒙工具打通全链路（自研）

不止测量，覆盖工程全生命周期

工程与构建 01

7 个工具

- harmony_project_profile
- harmony_schema_check
- harmony_build
- harmony_build_doctor
- harmony_lint
- harmony_sign
- harmony_toolchain_check

设备与模拟器 02

8 个工具

- harmony_devices
- harmony_emulator_list
- harmony_emulator_create
- harmony_emulator_start
- harmony_emulator_stop
- harmony_emulator_delete
- harmony_emulator_catalog
- harmony_image_check

装机与验证 03

6 个工具

- harmony_install
- harmony_launch
- harmony_uninstall
- harmony_logs
- harmony_device_test
- harmony_ui_regression

知识与检索 04

2 个工具

- harmony_api_lookup
- harmony_shell

通用 05

7 个工具

- read_file
- write_file
- edit_file
- list_dir
- grep
- find
- run_command

harmony_api_lookup　SDK ets/api 目录有上万份 .d.ts

一次性索引 → 缓存至 ~/.arkperf/apilkg.json → 用 SDKMtime 判断缓存是否过期 → 设安全上限 20000 文件 / 2000000 容符（超限即停并加误报告，防止扫描巨型目录卡死）

harmony_build_doctor　解析 hvigor 构建失败日志并归类根因

无法归类的如实输出 unknown，不乱猜——确定性写进工具行为，而非写在文档里。

23 个鸿蒙域工具 + 7 个通用工具：Agent 不只测得出，还装得上、跑得起来。

测量解耦 协议可换，判定可离线重判（测量层自研）

5 服务器 / 13 工具 · 4 类指标 · 三层结构

Layer 3 • Agent 调用层
内核封装调用工具，LLM 只做归因与表达

- 内核可整体替换 —— 换 Agent / 换模型不影响测量结论的有效性

Layer 2 • MCP 适配层
把测量函数包装成 MCP 工具，统一 JSON 输入输出

- 协议升级不触及测量逻辑
- 统一 **timeoutMs** 约定
- kernel → agent → cli 三层透传
  （agent 900s / cold_start 600s / memory·jank 300s / cpu 180s）
  ——原生 60s 扛不住约 200s 全量测量

Layer 1 • 测量逻辑层
纯 Python：采集 + 判定 + 统计（MW-U / MDD / 三奈）

- 原则：判定不依赖 Agent 语义
- 换协议 / 换内核 **不需要改**
- 可被 ***_test.py** 直接单测

为什么要换：**测量逻辑独立可判** —— 协议升级、内核替换都不影响测量结论的有效性。

三条收益（解耦换来的）

1. **测量逻辑独立可判**：验证阶段全靠 *_test.py，不依赖 Agent
2. **MCP 协议升级不触及测量逻辑**：协议与测量解耦
3. **原始序列随 JSON 落盘**：判定规则可离线重判，不用重测

5 个 MCP 服务器 / 13 个工具

agent(3) 编排	**cold_start(2)** 冷启动
memory(3) 内存	**cpu(3)** CPU
jank(2) 卡顿	

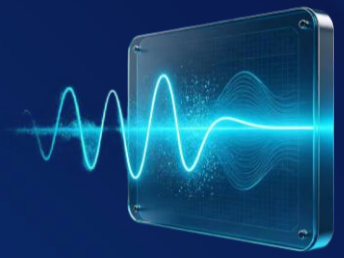
协议是可换的壳，测量与判定是搬不走的芯——所以判定能离线重判。

核心壁垒　卡点在测量与判定，不在 Agent

五重技术壁垒 · 逐页展开

抗噪声方法学 01

锚点归一化 / 内存三态判定 / 卡顿噪声度数，每项都有对照实验与确定性证据



→ P14 / P15

判定与采集分离 02

原始序列随 JSON 落盘，判定规则迭代可离线重判，不重测



→p16

官方口径内建 03

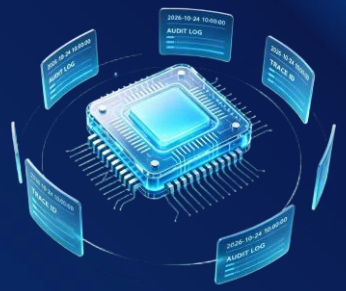
锚点 / 阶梯是官方推荐判据；卡顿只采系统判定；全程不自造阈值



→ P24 报告样例

工程诚实性 04

证据不足输出「未观测到」而非「正常」；8 条缺陷修复全记录留痕



→ P27/ P29

资产独立于内核 05

13 个工具以标准 MCP 协议装载，不绑定任何 Agent 实现



→ P12

规模证据： 23 个鸿蒙域工具 + 13 个 MCP 测量工具 · 5 个 MCP 测量服务器 · 自研 Go 内核 14 个模块 · 自研 Python 测量服务 15 个文件

做一个 Agent 不难，难的是让它测得准、判得对、还不瞎编——五重壁垒都在解这一句。

抗噪方法　一把尺子抵掉环境噪声

壁垒① · 锚点归一化

口径	会话1 → 会话2	若无归一化的结论
原始冷启动 P50	684 → 792.5 ms（+15.86%）	「代码退步 15.86%」❌ 假阳性
锚点（系统设置应用）	+21.9%	环境整体变慢
target/anchor 比值	-5.46%，p=0.85	无显著差异 ✅

举例　「尺子本身变长了」

用尺子量桌子：今天 68 cm，40 分钟后 79 cm。第一反应是「桌子变长了」——其实是尺子受热变了。再拿一个不变的东西量尺子：尺子涨 22%，桌子只涨 16%，那桌子其实变短了。

目标应用	系统设置	比值
= 桌子	= 标准件	= 真相

小结：目标应用 = 桌子，系统设置 = 标准件，比值 = 真相。

大白话：模拟器跑在电脑上，电脑一忙，测出来的启动时间就变慢——你没法判断是代码变差了还是电脑变卡了。办法是每测一次目标应用，紧接着测一次系统设置应用当「尺子」，用比值判断。

第一反应是假阳性

原始 P50 涨 15.86%，看上去像代码退步；其实是整台机器同时变慢，数字被环境拽着走。

尺子同步变慢

同一时间窗口，系统设置应用涨 21.9%——它没改过一行代码，涨的幅度就是环境噪声。

比值才是真相

target/anchor 相除得 -5.46%（p=0.85）：扣掉噪声后，目标应用不仅没退步，还略快了。

口径说明：冷启动降幅对外一律以归一化 -68.2% 为主数字，原始口径 -64.9%（-1031.5 ms）作为附注放在括号里——锚点归一化是本项目核心创新，主数字用归一化口径，叙事前后一致。

同一份代码、零改动，raw 口径能测出「退步 15.86%」；加一把尺子，结论变成「没变」。

显著铁律　小于 MDD，就不给结论

壁垒② · Mann-Whitney U + MDD

1 • 方法

Mann-Whitney U 检验（非参数，含并列秩 + 连续性校正）

$p < 0.05$ → 判为显著

报告指标：delta · pct · p_value · significant · MDD · verdict

2 • 为什么用非参数

不假设正态分布

适合冷启动带抖动的耗时数据

统计纪律：一律报「中位数 + 离散度 + p 值 + MDD」

3 • 纪律① 报告口径

一律报「中位数 + 离散度 + p 值 + MDD」，永不报单次绝对值

4 • 纪律② 预热与校验

预热 1 次丢弃

必须校验 StartMode==Cold，否则剔除

5 • 纪律③ 环境快照

主机 CPU / 内存 + 设备 uptime 随会话存储

6 • 纪律④ 三明治法

base1 → new → base2

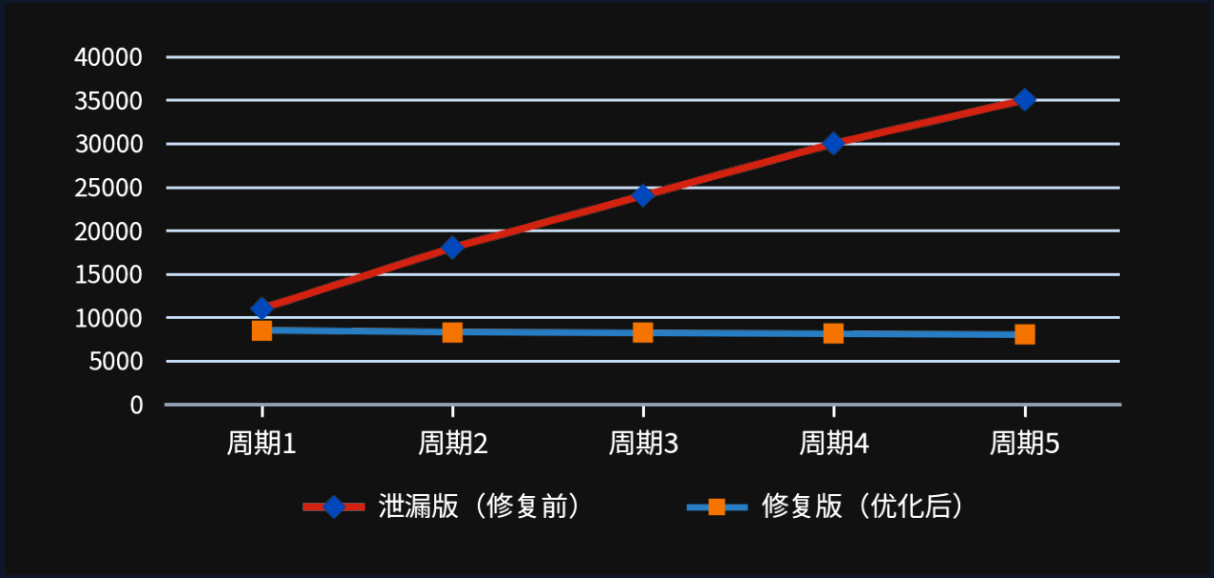
回滚校验，可选启用

比 MDD 还小的差异，我们不给结论

泄漏判定　强制 GC 后仍抬升，才叫泄漏

壁垒③ · 内存三态 + 强制 GC

真实证据：PerfLab 修复前后「强制 GC 后回落点」序列（kB）



参数：速率阈值 stable 512 / watch 2048 kB/min；单调性 0.7；三维度（PSS / ArkTS 堆 / Native 堆）分别判定。

读图：泄漏版逐周期抬高（阶梯 → ●）；修复版五周期稳定在 ~8.7 MB（锯齿 → ✓）。

采法：每周期 hidumper --mem-jsheap --gc（需 debug 签名）→ 采回落点 → uitest 驱动操作 → 采峰值。

✓ **稳定波动 = 锯齿 sawtooth**

正常

● **逐次抬升 = 阶梯 staircase**

泄漏

● **后段饱和 = 有界增长 capped**

缓存填充，非泄漏

大白话　一句话讲清判定逻辑

内存涨上去不一定是问题——缓存装满前也会一直涨。所以先强制回收一次，再看「每次回收后剩下的量」是不是一轮比一轮高。

一直抬高才叫泄漏；抬到一半平了，是缓存满了。

泄漏不是「涨了」，是「强制 GC 之后还在涨」——三态判定把缓存误判挡在门外。

数字全由代码算，AI 编不出（自研）

壁垒④ · 混合架构 · 覆盖 4 类问题，三级分级

规则 id	问题	ArkTS 级建议示例
slow_cold_start	启动耗时过长	非首屏初始化 TaskPool 异步化、首屏懒加载
memory_growth	内存泄漏	监听器/定时器注销、全局容器清理
memory_capped	有界增长	缓存上限（LRU）策略评估
cpu_high	CPU 占用过高	空转轮询消除、降频
main_thread_block	主线程阻塞	耗时任务搬离主线程（TaskPool）、拆分单帧渲染

规则引擎（确定性骨架）：**问题 → 原因 → ArkTS 级建议**

报告形态与设计取舍，集中在以下三点：

01 报告形态：五部分定式

固定五段骨架，Markdown 与 JSON 同步落盘

测试场景 / 关键指标 / 异常现象 / 可能原因 / 优化建议，五部分固定成式，便于横向对比与二次加工。

02 规则引擎：确定性骨架

五条规则命中即出三级分级建议，不经模型生成

问题 → 原因 → ArkTS 级建议，全部由代码算。规则库覆盖启动、内存、CPU、主线程四类高频问题，结论与建议均可追溯到具体规则条目。

03 为什么这样设计

不让 LLM 有造数的机会，而不是事后纠错

数字错了 AI 能发现，但 AI 编不出数字——幻觉免疫是让 LLM 没有造数的机会。建议有效性实证见 P21（五处修复与规约建议一一对应）。

让大模型「不碰测量」，比让它「少犯错」更可靠——它没机会编数字。

可迁移性　换掉整个内核，测量层没改一行

壁垒⑤ · 不侵入 + 解耦

指标	采集命令	数据来源
冷启动耗时	aa start -W	Ability Manager 启动回执
内存（ArkTS 堆 / PSS / Native）	hidumper --mem --jsheap --gc	系统内存快照
卡顿	hilog（JankFrameMonitor / RSJankStats）	系统卡顿监测日志
CPU	hidumper --cpuusage + 定时采样	进程 CPU 占用
交互注入	uitest uiInput	官方 UI 自动化
环境清理 / 状态查询	bm clean · uptime	包管理 / 主机负载

解耦要点

✓ **测量层走标准协议，不绑定内核**

13 个测量工具全部以标准 **MCP 协议**挂载，不依赖任何 Agent 实现；外墙已从开源框架整体替换为**自研 Go 内核**。

✓ **替换过程零适配，测量层一行未改**

mcp-servers / 15 个 Python 文件未做任何适配性修改，go.mod 无第三方 Agent 框架依赖（仅 bubbletea v2 / Wails3 / MCP Go SDK）。

✓ **现场可验证，可当场查代码**

能不动测量层就换掉整个内核，本身就是解耦成立的证明；评委质询时可直接查代码与依赖清单。

数据来源：官方 CLI / 不 root / 不改系统　·　换掉驱动内核，测量层一行未改

应用实测 真问题都被检出

自研 + 第三方 + 开源

	PerfLab（自研）	HarmonyStudy（第三方）	仿网易云（开源）
包名	com.example.perflab	com.wandroid.harmonyStudy	com.example.neteasy_cloud
冷启动 P50 (n=10 全 Cold)	1590.5 ms ⚠️ → 559.0 ms ✅	1771 ms ⚠️ 偏慢	481.5 ms ✅ 优秀
锚点比值 P50	1.37385 → 0.43685	1.468	0.408
内存 (5 周期+强制 GC)	🔴 阶梯 39956 kB/min → ✅ 锯齿	✅ 锯齿，ArkTS +136 kB/min 回落点稳定	✅ 锯齿，-13~+3 kB/min
CPU 空闲中位	—	1.42% ✅	0.95% ✅
主线程卡顿 (各 10 次注入)	8 次 🟡 → 0 次 ✅	0~4 次零星（噪声带内）	0~1 次（噪声带内）
检出结论	三类注入全部命中	启动耗时过长（真实检出）+ 三条归因方向 + ArkTS 建议	图片加载事故 → CPU -91.4%

口径公平性：两次会话的锚点 P50 分别为 1185 ms 与 1141.5 ms，仅差 3.7%——说明测量环境一致，跨应用比较成立。

自研靶子证明测得准，第三方与开源应用证明测得有用——HarmonyStudy 冷启动约为仿网易云的 3.6 倍。

靶子验证　三类坏味道，全部精准命中

案例 A · PerfLab 自研靶子

为什么**要自研靶子**：坏味道的量级是已知的——测出来对不对，可以逐项核对，这是验证测量工具**自身正确性**的办法。

指标	实测	设计注入值	命中
内存泄漏速率	39956 kB/min 🔴 阶梯	40 MB/min	✅ 误差 < 0.2%
主线程卡顿	8 次 / 6.3 次每分 🟡	持续阻塞	✅
冷启动 P50	1590.5 ms ⚠️	约 1700 ms	✅ 量级吻合

① 启动路径同步阻塞 1800 ms

Application 初始化阶段做**耗时同步调用**，启动主线程被占满，冷启动被硬性拉长，形成可复现的启动劣化。

② LeakPage 无界缓存 40 MB/min

页面级静态缓存**只写不清**，泄漏速率恒定可控，用于核对泄漏量级、增长曲线与归因准确性。

③ BlockPage 主线程周期阻塞

按固定节奏在主线程**做重计算**，形成可预期的卡顿频次与时长，用于核对卡顿检出与分级能力。

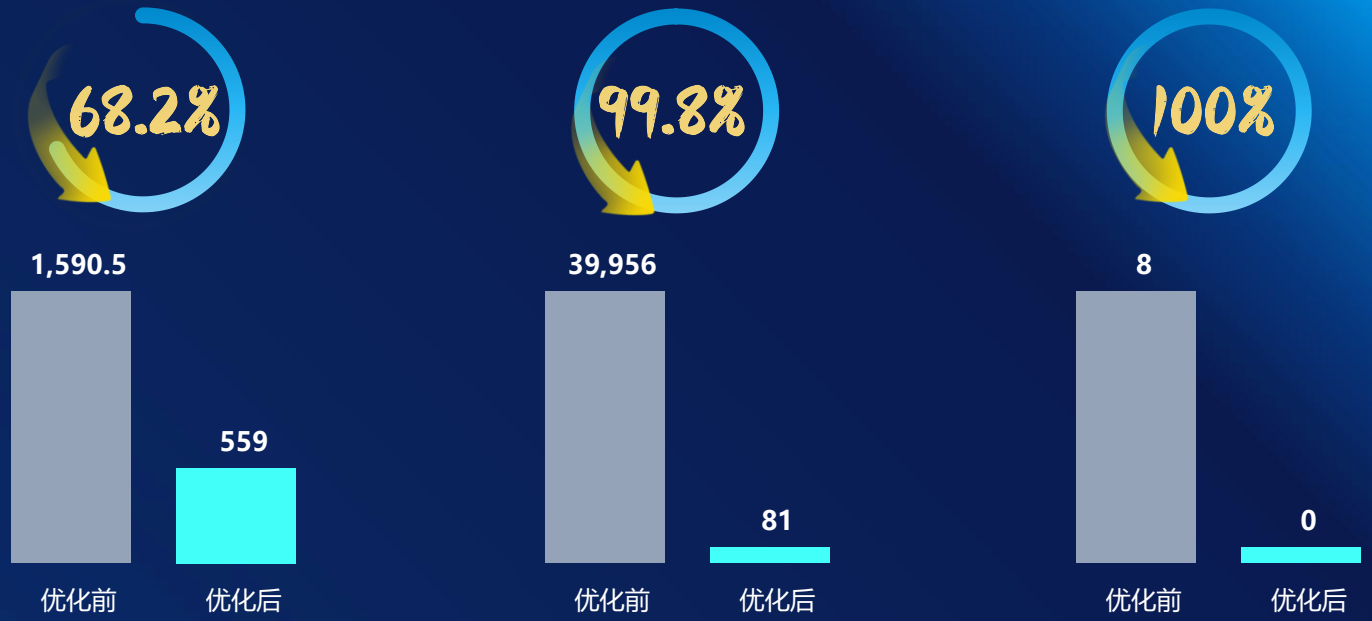
测量条件　设备 emulator 6.1.0.126 / API 24（常驻，两轮测量间未重启）；基线 2026-09-21 23:05~23:32（build #1，含坏味道）；优化 2026-09-22 00:25 重装后 00:40~01:02（build #2，已修复）；同设备、同参数、同采样数。

场景直达　aa start --ps scene leak|block|discover，配合 SceneHolder 静态类中转（绕开 .ts ↔ .ets 的 import 边界），全程无人工 UI 导航。

三类注入、三种量级，全部被检出——先证明工具测得准，再拿去测别人的应用。

判定翻转　三项结论全翻，闭环成立

案例 A · 建议 → 修复 → 复测



归一化比值
0.43685
1.37385 → 0.43685

冷启动原耗时
-64.85%
1031.5 ms

冷启动归一化明细

n = 10/10

P50 耗时：1590.5 ms → **559.0 ms**
样本量：n = **10/10**（全部 StartMode = Cold）
归一化比值：1.37385 → **0.43685**　锚点 P50：1197.5 ms

关键数据　冷启动原耗时 = **1031.5 ms**（**-64.85%**）

结论稳健性 + 卡顿对照

Mann-Whitney U

p = **0.0002** < 0.05；MDD = **16.0%**，改善幅度约为 MDD 的 **4.3 倍**
★ 锚点归一化佐证：环境明显变慢，归一化后仍得 **-68.2%**
注入 **10** 次 · 修复前 **8 次 / 6.3 每分** · 修复后 **0 次**

判定总结

三项全翻

五处修复：启动路径同步阻塞异步化 / LeakPage 无界缓存加上限 / BlockPage 主线程阻塞剥离
判定翻转 ● 缓慢 → **优秀**　● 防漏 → **很缓**　● 轻度 → **无卡顿**
环境漂移：两轮 CPU 负载 15.4% → 36.7%；锚点漂移 +6%，归一化口径已对齐

建议 → 修复 → 复测：三项判定全部翻转，闭环不是说法，是可复核的数字。

事故定位 107 宽坑位加载 17.9 MB 巨图

案例 B · 仿网易云（开源）

com.example.neteasy_cloud（仿网易云音乐，开源 ArkTS 项目本地编译）

开源出处：github.com/linwu-hi/open_neteasy_cloud

位置	显示尺寸	原图实测
新碟封面	107 宽	36771 × 27002 像素 / 17.9 MB ⚠
新碟封面	107 宽	4000 × 4000 / 3.4 MB
推荐歌单封面	107 宽	最大 1024 × 1024 / 88 KB
MV 封面	337 × 213	800 × 800 / 62 KB

「发现」页 14 处网络图片全部原图直链

典型「未按显示尺寸请求」事故：

107 宽的坑位，下载了 17.9 MB 巨图——带宽、解码、内存三重代价。

方法学贡献

Image 磁盘缓存会掩盖加载成本，测量前必须 rm clean -n ＜bundle＞ -c 统一清缓存口径。

脱坑记录：首次测量 PSS 很平稳、微图发现图片早已被命中。

工程适配

该项目因为 API 9，为本次测量升级到 API 24（编译错误 10+2 处，用 新增 / 显式类型 Record 转换修复，装机验证通过）——测量前做了工程适配，不是拿来即用。

优化方向

改为按显示尺寸请求 CDN 缩略图（?param=WxH）——效果见下一页。

14 处原图直链、最大一张 17.9 MB，塞进 107 宽的坑位——问题具体到行，才改得动。

修复成效　改一处，三个维度同时改善

案例 B · 14 处改动 / 8 个唯一 URL



系统级发现

Image 组件按原图尺寸解码并常驻内存：**4000 × 4000 → 64 MB** 位图；优化后 **400 × 400 → 0.64 MB**（按显示尺寸请求 CDN 缩略图 ?param=WxH）

这不是应用层的 bug，是框架行为——调出来才有资格修

诚实记录

主线程卡顿：**0 次 → 0 次**　两轮均为 0，该应用本底健康；优化收益不体现在此维度（如实记录，不凑收益）

修复方式：改为按显示尺寸请求 CDN 缩略图（?param=WxH）

一个「请求原图」的坏习惯，同时吃掉 CPU、内存和流量——改一处，三条线一起降。

报告形态　五段结构，与赛题逐条对齐

open_neteasy_cloud 优化前后对比报告（真实开源应用案例）

- 被测应用：`com.example.neteasy_cloud` / EntryAbility（仿网易云音乐开源 ArkTS 项目）
- 测量场景：`--scene discover` 场景直达 IndexPage"发现"Tab（自建 scene 路由，见方法学）
- 设备：emulator 6.1.0.126(SP1DEVC00E120R4P11) / API 24（模拟器常驻，未重启）
- 基线：2026-09-22 13:21 安装（Build A，原图直链）
- 优化：2026-09-22 18:01 安装（Build B，14 处网络图加 CDN 尺寸参数）
- 口径：每轮测量前 `bm clean -n <bundle> -c` 清 Image 磁盘缓存（统一冷缓存，见方法学）

一、优化内容

"发现"页 14 处网络图片全部使用**原图直链**（网易 CDN），改为按显示尺寸请求缩略图（`?param=WxH`，网易云 CDN 官方缩放参数）：

位置	显示尺寸	数量	请求尺寸	原图实测
我的-歌单封面	56×56	1	200y200	153KB
Swiper banner	349×145	3	700y290	1080×420 / 31KB
推荐歌单封面	107 宽	6	400y400	最大 1024×1024 / 88KB
新碟封面	107 宽	3	400y400	**36771×27002 / 17.9MB**、4000×4000 / 3.4MB 等
MV 封面	337×213	1	800y800	800×800 / 62KB

下载量实测（8 个唯一 URL，主机侧）：**21.9MB → 518KB（-97.6%）**。其中"新碟"第一张原图为 36771×27002 像素的 17.9MB 巨图，属典型的"未按显示尺寸请求"性能事故。

二、总体结果

维度	优化前	优化后	变化
加载期 CPU 占用（中位）	11.88% 🟡 轻度	1.02% ✅ 低	**-91.4%，判定翻转**
内存稳态 PSS	156.8 MB	56.6 MB	**-63.9%（-100.2 MB）**
下载量（8 个唯一 URL）	21.9 MB	518 KB	**-97.6%**
主线程卡顿	0 次 ✅	0 次 ✅	不变（健康本底）

一、测试信息

包名 / Ability / 场景 / 设备 / 采集时间 / 采集模块

二、测试场景与关键指标

逐场景状态 + 关键指标表（58.0 s 采集 · 🔴 阶梯 39956 kB/min）

三、异常现象、可能原因与优化建议

按严重度分级（✅ 🟡 🟠 🔴），每条带证据

四、总体结论

一句话总结

五、说明

绝对耗时受环境影响，以稳健统计量 + 阈值为准

对齐赛题要求：赛题要求报告包含「测试场景、关键指标、异常现象、可能原因、优化建议」——我们的报告形态与之逐条对应。

可复跑：全部结果可复跑、可核验：报告 MD + JSON 双落盘，原始序列可复算，会话审计逐轮可回放。

因错施策　该重试重试，不该重试立即停

错误分类 · 五类分流

错误类型	策略	理由
网络层故障 / 408 / 429 / 5xx	指数退避，最多 6 次	瞬时故障，重试有效
服务端给 Retry-After	听它的（钳制 120 秒）	尊重服务端限流指示
其他 4xx	不重试	密钥错、模型名错——重试一百次还是同样的错
本地超时	不重试	确定性错误，重试只让用户多等
流已开始吐字后中断	不重试	否则同一段话会再打一遍

设计逻辑：

重试只对「瞬时故障」有意义；确定性错误重试只会放大等待。指数退避最多 6 次，与业界成熟 Agent 框架行为对齐。

底座：所有 hdc / hvigorw / ohpm 调用统一走 execx——超时、输出截断、Windows 批处理包装（hidewindow 专项处理）、错误分类集中处理，「数据不出错」的底座。

工具使用纪律

（写进系统提示词的硬约定）

- 工具报错就如实转述，不要编造成功
- 结论必须带证据：引用工具输出的原文
- 拿不到数据就说拿不到。
  宁可少说，也不要给一个编造的数字
- 同一工具同样参数失败两次后换策略，不重试第三次

Agent 的体验崩塌多数不是「算错」，而是「卡住、重复输出、假装成功」——这三件事都由错误分类与重试策略控制。

把「不该重试」认出来，比把「重试」做得多更值钱——它省的是用户的等待和信任。

能力边界　没发现问题，不等于没测到

5 条被动局限 + 2 条主动取舍

局限	影响	缓解
模拟器镜像帧率能力缺失（恒返回 0）	无帧率指标	卡顿计数替代；真机可启用 SP_daemon
aa start -W TotalTime 不含首帧（系统性偏低约 33%）	绝对值偏低	做差时两边同时偏可约掉；只用于对比
锚点口径 MDD 约 19%	小幅优化可能测不出	增大采样数（n=40 → 约 9.6%）/ 三明治法
jank 信号弱（0~2 次/窗口）	轻度卡顿难判定	噪声底数防误报；重载加大注入
内存强制 GC 需 debug 签名	release 应用判定偏保守	自动降级 + 告警

被动局限（环境与工具的客观约束）

主动取舍 · 卡顿不自造阈值

只采信系统自身判定并计数；同条件复测 0↔1 翻转 → 设 JANK_NOISE_FLOOR=1，≤1 次不构成结论。

主动取舍 · 砍掉时延数值

INTERACTION_*_LATENCY 被 fling 惯性动画污染（E2E 抬到秒级）、订阅被拖垮、并发 hdc 冲突、时钟偏移——四坑同路，收窄为「判定有无卡顿」。

「没发现问题」和「没测到」是两件事——混为一谈会让用户误以为优化有效。所以我们把边界写在报告里，而不是藏在心里。

主动说清测不到什么，比多报一个数字更能建立信任。

缺陷记录 8 条误判，每条驱动一次修复

09-21 ~ 09-22 · 全记录留痕

日期	缺陷	修复
09-21	内存判定用首尾差，双向误判（一次性分配误报、真泄漏漏报）	升级为周期式锯齿 / 阶梯判据
09-21	不驱动操作测不出泄漏（方法层假阴性）	uitest 操作驱动 + 周期采样
09-21	卡顿 count=1 即判 blocked（0↔1 翻转）	噪声底数 ≤1 不构成结论
09-21	MCP 工具 60 秒超时不够全量测量	框架加每服务器 timeoutMs 三层透传
09-21	MCP 服务器加载失败（依赖缺失）	隔离 venv + 三级 import 兼容
09-21	开源项目 API 9 → 24 编译错误 12 处	! 断言 / 显式类型 / Record 转换
09-22	冷启动 compare 测错对象，产出 -32.66% 假阳性	三道防线：基线对象恢复 / 口径断裂 fail-fast / 锚点警告
09-22	有界缓存填充期误判泄漏（2082 kB/min 踩线 🔴）	capped 三态识别，回归验证 6/6

整体来看，泄漏判定坚持**三种内存口径交叉验证**：判断三条同时出现——ArkTS 堆断档 / PSS 阶差 / Native 堆缓慢上升，单一口径的偶发波动不会被判定为泄漏。

三级跳演进

首尾差

一次性定位生死；误判双向

斜率 + 弹性

引入趋势判断，仍依赖阈值

周期式强制 GC + 弱引用/阶梯有异态

当前：三维强分别判定

每一级都由真实误判驱动

8 条缺陷全是从「判错了」里来的——记录它们，比宣传零缺陷更可信。

代码结构 1.6 万行 Go，14 个内核模块

104 文件 · 分层清晰 · 全部有测试

Go 代码文件
104 个文件
非测试 56 / 测试 48

Go 非测试代码
约 15,850 行
1.6 万行量级

Python 代码
15 / 3,735 个文件 / 行
mcp-servers 测量服务

Go 测试函数
475 个
全部模块有测试覆盖

目录结构（104 文件 · 分层清晰 · 全部有测试）

cmd/akperf	CLI 入口	入口
internal/kernel	内核层：模型对话、工具注册、会话、配置	内核
internal/app	装配层	装配
internal/tools	本地工具（跨域、通用）	工具
internal/skill	技能层：技能加载、说明	技能
internal/mcp	MCP 客户端	MCP
internal/domain/harmony	鸿蒙工具链探测与工程解析	鸿蒙
internal/frontend/tui	TUI 界面	界面
desktop	桌面端（Wails3 + Vue3）	桌面
mcp-servers	自研 Python 测量服务	测量
skills	内置技能	技能

规模的意义

1. 内核 14 个模块 + 测量层 15 个文件，全部自研并有测试覆盖
2. 数字均由 find / grep / wc 对当前代码实测得出，可当场复现
3. 对外可用「15,800 行 / 3,735 行」量级表述

代码量级一览

14 个内核模块 · 104 个 Go 文件 · 475 个测试函数 · 分层清晰、边界清楚

1.6 万行 Go 不是堆出来的量，是 14 个模块各带测试、边界清楚的量。

工程规范　代码可测、结果可验、仓库可复现

go test ./... · 工具可脱 Agent 单跑

① 475 个测试　go test ./... 一键全量

类别	覆盖
内核	模型调用循环、重试策略、流式输出、会话管理、审批规则
工具	注册完整性、审批标注、路径作用域
鸿蒙域	hdc 参数顺序、设备列表解析、模拟器配置、构建日志诊断
前端	会话切换、斜杠命令、补全菜单、粘贴处理、流式渲染
MCP	连接管理、配置加载、输出目录解析

② 交付物清单

类型	文件
架构设计文档	architecture.md
测试报告	test-report.md
使用说明	mcp-servers/README.md
优化对比报告 ×2	perflab / nyc optimization-comparison
场景报告 ×11	experience-report-*.md / .json
会话原始数据	perflab_cold_base / opt.json（锚点比值可复算）

③ 开源与独立使用

项目	内容
开源地址	github.com/fan-weibo/ArkPerf
绑定关系	工具集不绑定 Agent，可不经 LLM 直达执行
独立命令 ①	python agent_test.py <包名> --quick
独立命令 ②	python cold_start_test.py measure -o base.json
独立命令 ③	python cold_start_test.py compare base.json

475 个测试守住「不退化」，脱 Agent 直跑守住「可复用」——两件事一起才叫交付。

赛题响应 基础 6 条全达，3 条超要求

挑战目标 4 条达成、1 条部分

基础目标 · 6 条全部达成

#	赛题基础要求	达成证据	状态
1	采集 ≥3 类基础体验指标	4 类：冷启动 / 内存 / CPU / 卡顿	✅ 超出
2	识别 ≥3 类典型体验问题	4 类：启动慢 / 泄漏 / CPU 高 / 阻塞，含三级分级	✅ 超出
3	完整 Agent 工作流（包名/工程路径）	端到端 8 轮 9 调用 2 分 31 秒自主完成	✅
4	规则引擎/大模型生成结构化报告	ISSUE_RULES + LLM 混合，MD / JSON 双落盘	✅
5	至少一个示例应用 + 前后对比	三应用：PerfLab + HarmonyStudy + 仿网易云	✅ 超出
6	架构文档 / 使用说明 / 测试报告 / 演示材料	见 P29交付物清单	✅

挑战目标 · 4 达 1 部分

#	挑战目标	状态	说明
1	读 ArkTS / ArkUI 代码定位问题	◐ 部分	测量证据驱动归因；DevEco 集成列远期规划
3	多轮诊断闭环 + 自动差异报告	✅	baseline → 优化 → compare → 三明治法
4	可插拔大模型接口	✅	任意 OpenAI 兼容厂商
5	对话式开发者问答	✅	TUI / 桌面端多轮上下文
6	向社区工具孵化可行性	✅	MCP 标准协议，任意 Agent / CI 可挂载

基础 6 条全达、3 条超要求；挑战目标 4 条达成、1 条部分

团队分工 覆盖内核、测量与实测

01 **范伟博**

队长 · 核心技术负责人

主导技术方案落地与核心功能实现，保障项目技术壁垒与可交付性

02 **肖美儒**

项目内容策划

统筹研究逻辑与文案框架，确保汇报内容与评审评分标准精准对齐

03 **吴燕婷**

视觉设计

负责答辩视觉体系搭建与信息可视化呈现，提升评审沟通效率

04 **黄育钊**

指导老师

负责技术方向把关与专业指导，把控研究路线与成果质量。

内核、测量、实测三条线各有主责，方法学与报告交叉复核。

从「发现问题」到「持续进化」
让体验优化成为闭环

以智能 Agent 之力，重塑鸿蒙应用体验

感谢聆听 · 恳请各位评委批评指正

仓库：github.com/fan-weibo/ArkPerf

一句话进去，一份带统计证据的报告出来——这就是 ArkPerf。

备用 A1　PerfLab 修复明细表

位置	坏味道	修复
EntryAbility.onCreate	burnSync(1200) 同步阻塞启动路径	@Concurrent + taskpool.execute 异步化
Index.aboutToAppear	burnSync(600) 阻塞首页挂载	taskpool.execute 异步化
LeakPage	全局容器无界累积 + 销毁不清定时器	MAX_CHUNKS=4 上限 + shift() 淘汰 + 销毁清理
BlockPage	定时器主线程 burnSync(500)	搬离主线程 + 销毁清定时器

四行改动，对应三项判定翻转——修复是可指认的。

13 个 MCP 工具完整清单

agent(3) 端到端编排入口	· experience_analysis	· experience_compare	· experience_report
cold_start(2) 冷启动测量与对比	· cold_start_measure	· cold_start_compare	
memory(3) 内存快照 / 趋势 / 三态判定	· memory_snapshot	· memory_trend	· memory_classify
cpu(3) CPU 占用 / 趋势 / 判定	· cpu_usage	· cpu_trend	· cpu_classify
jank(2) 卡顿测量 / 判定	· jank_measure	· jank_classify	

5 个 MCP 服务器 / 13 个工具　全部以标准 MCP 协议挂载，不绑定任何 Agent 实现。

备用 A3　生态位补充：补的是哪三层

层	能力	OpenHarmony 北向
①	采集数据	hdc 官方 CLI ✅
②	自动判定	◐ Hypium-perf 支持指标分析和回归监控，但无统一阈值断言 API
③	自动报告	◐ perfdata 需额外解析，无标准 JSON 输出
④	自动前后对比	⚠️ 需自建 Baseline 对比机制

② 自动判定 — P15、P17

规则引擎 ISSUE_RULES 五条
三级严重程度分级
Mann-Whitney U 检验
MDD（最小可检测差异）

③ 自动报告 — P24

Markdown + JSON 双落盘
报告采用固定五段结构

④ 自动前后对比 — P21、P23

baseline /optimized tag 归档
三明治法回滚校验

四行改动，对应三项判定翻转——修复是可指认的。